

CHAPTER 4

REQUIREMENT ANALYSIS AND DESIGN

4.1 Introduction

This chapter describes the system design that has been developed based on the requirements from Chapter 3. It starts with the use case analysis that defines how users of each role will behave in the system. The next step involves describing the architecture design for the system, which includes such aspects as the network topology within AWS, security control setting, IAM policy design, and DevSecOps pipeline design. Next comes the database schema design that includes PostgreSQL table design and row-level security policies that ensure proper data isolation at the storage level.

The requirement analysis is described by user stories and use cases in Section 4.2. The design of the whole project involving system architecture is provided in Section 4.3. Database design and its schema will be provided in Section 4.4. Interface design for all users is given in Section 4.5. Finally, Section 4.6 provides a summary of the chapter.

4.2 Requirement Analysis

This section documents the functional requirements of the proposed system as derived from the stakeholder discussions conducted at Alamin Clinic, described in Section 2.2.3. The requirements are presented through three complementary representations: a use case diagram that maps the relationships between each actor and the operations they are authorised to perform, a set of user stories that express requirements from each actor's perspective in Agile format, and detailed use case specifications that define the precise interaction behaviour expected at each system entry point.

4.2.1 Use Case Overview

The system supports three types of users including: Doctor, Admin, and Patient, each with their own set of allowable operations. Figure 4.1 illustrates the use case diagram that includes all actors along with their allowable operations in the system.

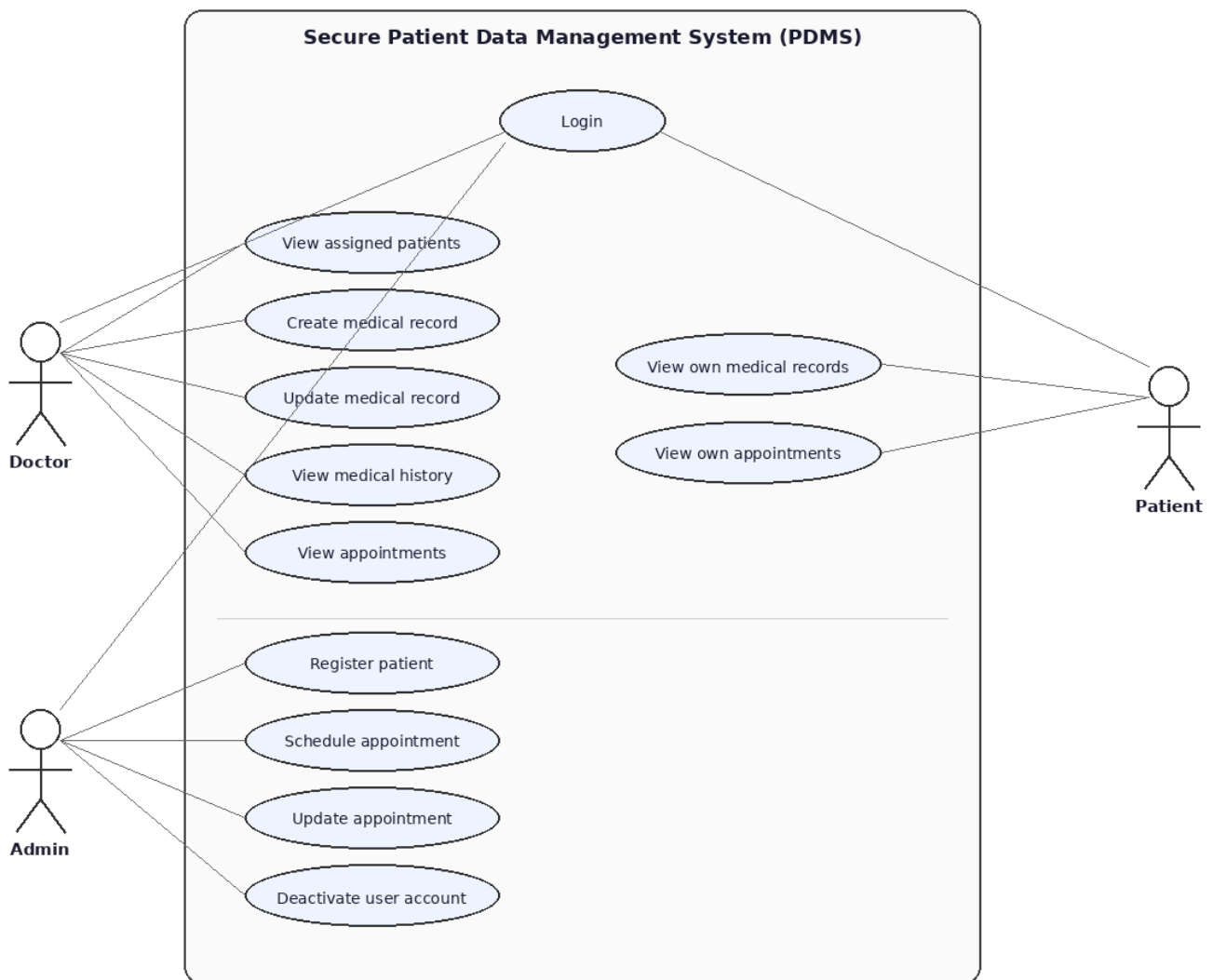


Figure 4.1 UML Use Case Diagram: Secure Patient Data Management System

4.2.2 User Stories

User stories capture the system's functional requirements from the perspective of each actor, following the Agile format: "As a [role], I want to [action], so that [benefit]." Table 4.1 presents all eighteen user stories, organised by module. These stories were derived directly from the stakeholder interviews conducted in Section 2.2.3 and informed the use case design in Section 4.2.3.

Table 4.1 User Stories by Module

ID	Role	User Story	Module
US-05	Admin	As an Admin, I want to deactivate user accounts, so that former staff or inactive patients can no longer access the system.	Auth
US-06	Admin	As an Admin, I want to register new patients and assign them a treating doctor, so that their records can be managed securely from the point of registration.	Patient Mgmt
US-10	Doctor	As a Doctor, I want to create medical records for my assigned patients, so that I can document diagnoses and prescriptions in a secure, auditable system.	Medical Records
US-11	Doctor / Patient	As a Doctor, I want to view records of my assigned patients; as a Patient, I want to view my own records in read-only mode to ensure data is accessible.	Medical Records
US-14	Admin	As an Admin, I want to schedule appointments linking a patient to a doctor at a specific date and time, so that consultations are organised.	Appointments

The complete set of eighteen user stories covering all four modules is provided in Appendix B, Table B.1.

4.2.3 Use Case Descriptions

Table 4.2 lists all eighteen use cases for the proposed system, organised by functional module.

Table 4.2 System Use Case Directory

UC ID	Use Case Name	Actor	Summary Description
UC-01	User Login	Doctor, Admin, Patient	Authenticate via credentials; issue JWT via <code>httpOnly</code> cookie.
UC-02	User Logout	Doctor, Admin, Patient	Terminate session; clear JWT cookie and write audit log.
UC-03	Account Lockout	System, Admin	Automatic lock after 3 failed logins; emit CloudWatch alert.
UC-04	Admin Creates User	Admin	Create user with role and link corresponding profile row.
UC-05	Admin Deactivates User	Admin	Soft-delete user via active flag; preserve records, block login.
UC-06	Register New Patient	Admin	Register demographics and doctor; generate temp credentials.
UC-07	View Patient Profile	Doctor, Admin	View demographics; Doctors restricted to assigned patients.
UC-08	Update Patient Info	Admin	Update demographic details and record changes to audit log.
UC-09	Assign Doctor to Patient	Admin	Reassign doctor; changes visibility boundary via PostgreSQL RLS.
UC-10	Create Medical Record	Doctor	Doctor writes clinical record for assigned patient; logs action.
UC-11	View Medical Record	Doctor, Patient	Doctor views assigned patients; Patient views own data via RLS.
UC-12	Update Medical Record	Doctor	Doctor modifies record they authored; RLS blocks cross-updates.
UC-13	View Medical History	Doctor	Doctor views chronological clinical history of assigned patient.
UC-14	Schedule Appointment	Admin	Book appointment; run conflict validation checks before confirmation.
Continued on next page			

Table 4.2 – Continued from previous page

UC ID	Use Case Name	Actor	Summary Description
UC-15	View Doctor Schedule	Doctor	Doctor views own upcoming schedule; cross-doctor requests rejected.
UC-16	View Own Appointments	Patient	Read-only patient access to own appointments; actions disabled.
UC-17	Update Appointment	Admin	Edit appointment; rerun conflict validation checks on new time slot.
UC-18	Cancel Appointment	Admin	Set appointment status to 'cancelled'; retain row for auditing.

4.2.4 Sequence Diagrams

Sequence diagrams for the four primary use cases are presented in Appendix G (Figures G.1–G.4). Each diagram traces the complete request path from the user's browser through the React frontend, Express API, and PostgreSQL database. Figure D.1 (UC-04 Login) illustrates the bcrypt verification and JWT issuance flow. Figure D.2 (UC-01 Create Medical Record) shows the three-layer authorisation chain: RBAC middleware, application-layer ownership check, and PostgreSQL RLS. Figure D.3 (UC-02 Schedule Appointment) demonstrates the conflict detection logic. Figure D.4 (UC-03 Patient Views Records) shows the RLS read-path enforcement.

4.3 Project Design

This section presents the complete design of the proposed Secure Cloud-Based Patient Data Management System. The design is organised across five components: the overall three-tier system architecture deployed within an AWS Virtual Private Cloud, the VPC network topology and subnet configuration, the security controls applied at the network and identity layers, the six-stage DevSecOps CI/CD pipeline, and the user interface wireframes for each of the three user roles. Together, these components constitute the full technical blueprint that will guide implementation of the system.

4.3.1 System Architecture Overview

The designed architecture for the system uses a three-tiered web application running on the AWS Virtual Private Cloud network. The presentation layer, application

layer, and data layer of the system have been designed to be present in separate layers of subnets with their respective access policies. There is no direct connection between the presentation layer and the data layer. Figure 4.2 Shows the system architecture.

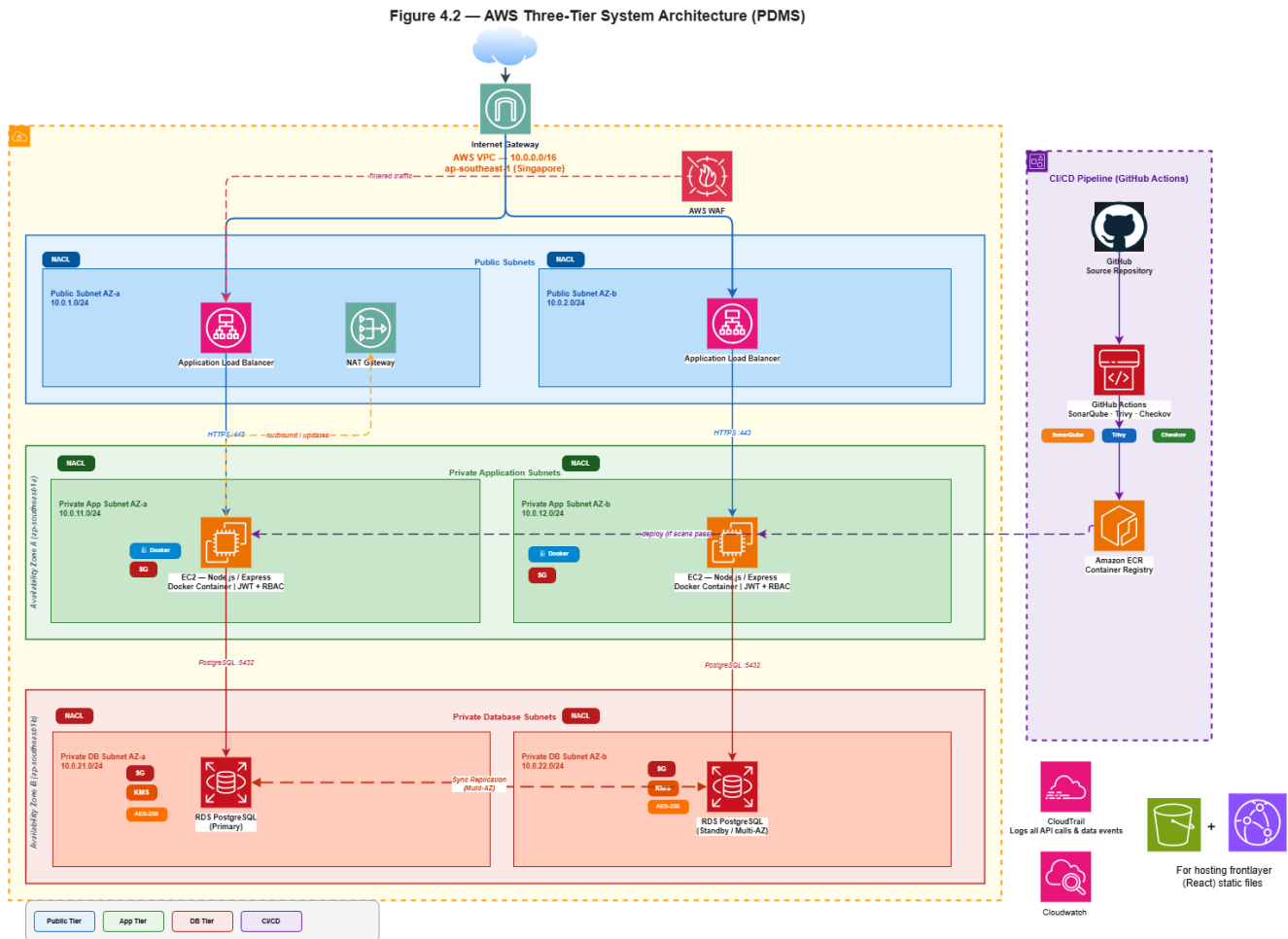


Figure 4.2 AWS Three-Tier System Architecture for the Patient Data Management System (PDMS)

4.3.2 VPC Network Design

The system is deployed within a single AWS VPC with the CIDR block '10.0.0.0/16'. The VPC is divided into six subnets across two Availability Zones with three subnet tiers per AZ to achieve high availability and fault isolation. Table 4.3 shows the subnet configuration for each layer.

Table 4.3 VPC Subnet Configuration

Subnet	CIDR	Availability Zone	Tier	Purpose
public-subnet-a	10.0.1.0/24	ap-southeast-1a	Public	ALB, NAT Gateway
public-subnet-b	10.0.2.0/24	ap-southeast-1b	Public	ALB (multi-AZ)
app-subnet-a	10.0.3.0/24	ap-southeast-1a	Private	EC2 application instances
app-subnet-b	10.0.4.0/24	ap-southeast-1b	Private	EC2 application instances
db-subnet-a	10.0.5.0/24	ap-southeast-1a	Isolated	RDS primary instance
db-subnet-b	10.0.6.0/24	ap-southeast-1b	Isolated	RDS standby instance (Multi-AZ)

Routing configuration. The public subnets are bound to the route table which sends the traffic destined for ‘0.0.0.0/0’ through the Internet Gateway, thus allowing internet traffic directed to the ALB and outbound internet traffic from NAT. The application subnets are bound to the route table which sends the traffic destined for ‘0.0.0.0/0’ via the NAT Gateway, thus allowing EC2 instances to establish outbound connections (to AWS API, package repositories, etc.), but not being accessible from the internet itself. Database subnets have no connectivity to the internet at all their route table consists of only one route which is the local VPC route (‘10.0.0.0/16’).

4.3.3 Security Group Configuration

Security Groups act as virtual firewalls at the instance level, enforcing allow-list rules for inbound and outbound traffic. the complete inbound and outbound rule definitions for all three Security Groups are provided in Appendix F, Table F.1.

4.3.4 Network Access Control List (NACL) Configuration

Network ACLs provide a stateless secondary security boundary at the subnet level, complementing Security Groups. Three NACLs are defined, one per subnet

tier. The full NACL rule sets for all three subnet tiers are provided in Appendix F, Table F.2.

4.3.5 IAM Policy Design

The three IAM roles and their associated permission boundaries are defined in full in Appendix F, Table F.3. The governing principle across all three roles is least privilege.

4.3.6 DevSecOps Pipeline Design

The CI/CD pipeline is implemented in GitHub Actions and is triggered on every push to the ‘main’ branch. The pipeline executes six stages in sequence. A failure at any stage with a critical finding terminates the pipeline and the deployment does not proceed. Figure 4.3 shows the pipeline flow diagram, furthermore, Table 4.4 details how a failure at any scan stage blocks the pipeline and prevents the deployment from proceeding.

Figure 4.3 — CI/CD Pipeline: Shift-Left Security (GitHub Actions)

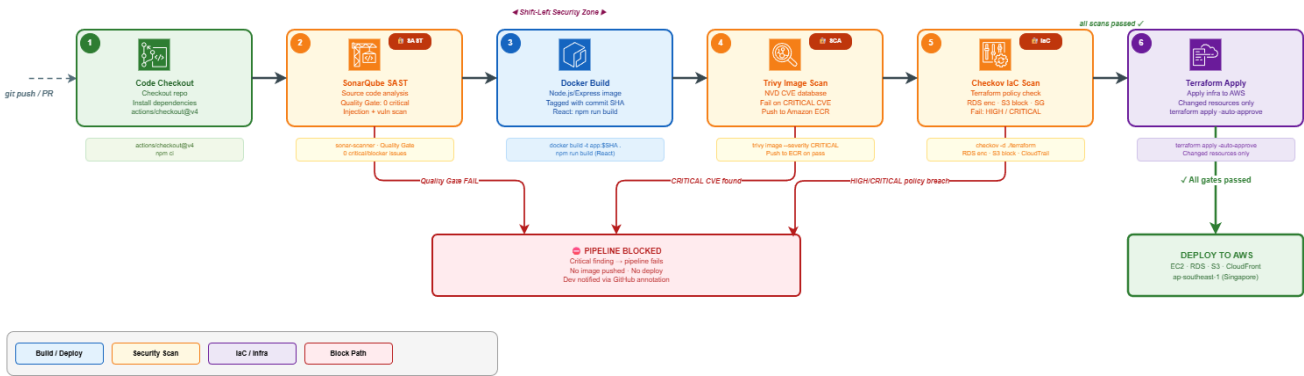


Figure 4.3 CI/CD Pipeline with Shift-Left Security Gates (GitHub Actions)

Table 4.4 DevSecOps Pipeline Stages Summary

Stage	Stage Name	Core Tool	Target Artifact	Failure / Gate Condition
1	Code Checkout	GitHub Actions	Source Code	Dependency resolution or fetch failure
2	SonarQube SAST	SonarQube Scanner	Application Code	Quality Gate failure (Critical/Blocker)
3	Build	Docker / npm	Container / Static	Compilation or image assembly failure
4	Trivy Image Scan	Trivy	Backend Image	Any CRITICAL severity CVE finding
5	Checkov IaC Scan	Checkov	Terraform Files	Any HIGH or CRITICAL policy violation
6	Terraform Apply	Terraform CLI	AWS Infrastructure	Cloud provider API or syntax execution error

4.3.7 Application Layer Class Design

Figure 4.4 presents the class diagram for the Node.js/Express application layer, showing the six data model classes and the two service classes that together form the backend of the system.

The six model classes ('User', 'Patient', 'Doctor', 'MedicalRecord', 'Appointment', and 'AuditLog') correspond to the database schema provided in Section 4.4. They represent their respective database tables in the application with static methods for table query and manipulation. The two service classes ('AuthController' and 'RBACMiddleware') are implemented at the request handler level. 'AuthController' is responsible for managing the processes of logging in, logging out, and validating the

JWT token. ‘RBACMiddleware’ handles all incoming requests and ensures the caller meets RBAC requirements before reaching the handler.

Figure 4.5 — UML Class Diagram: Patient Data Management System

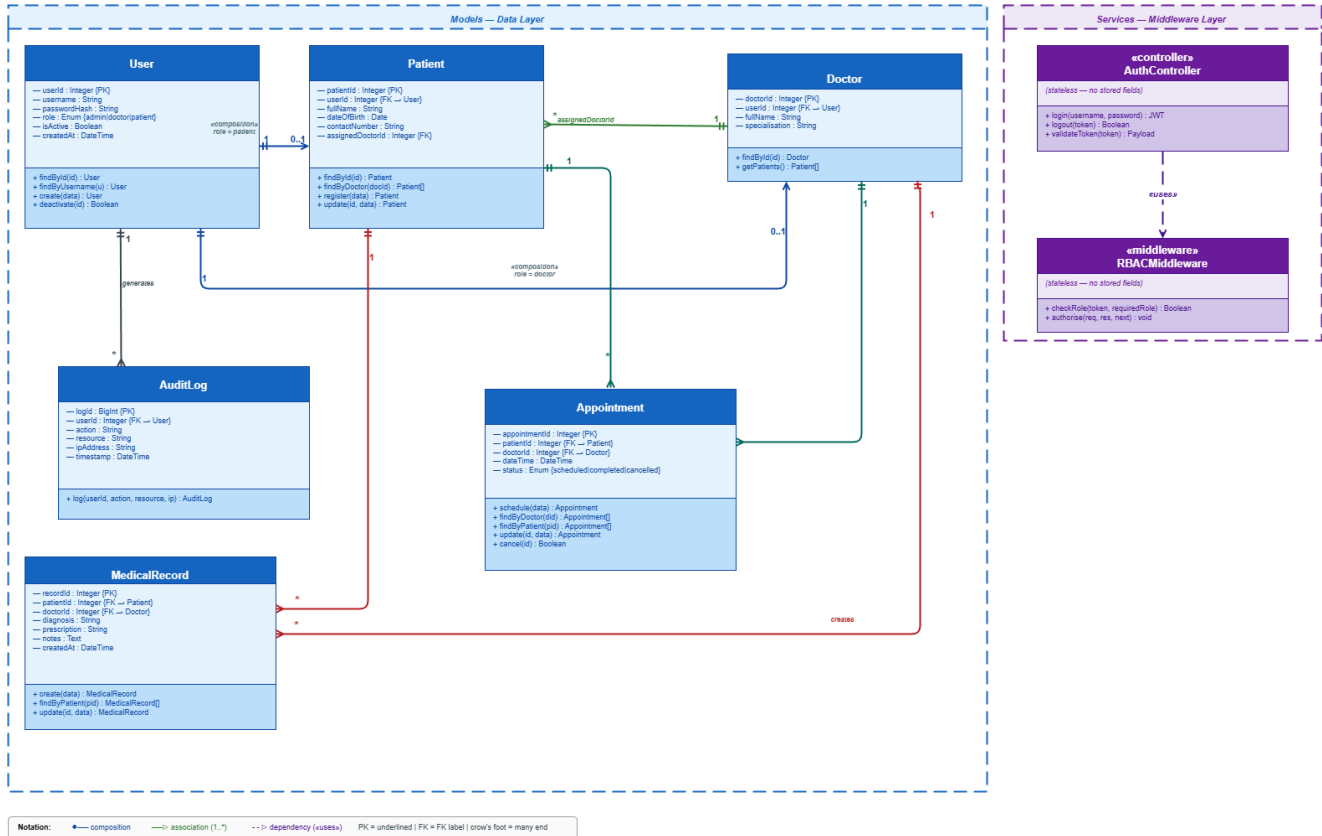


Figure 4.4 UML Class Diagram of the Patient Data Management System

4.3.8 Security Design

This system utilizes a layered security approach spanning five layers. Authentication uses bcrypt hashing for passwords (with cost of 12) and the generation of JSON Web Tokens as cookies which have the attributes `httpOnly`, `Secure`, and `SameSite=Strict`, with account lockout after three failed attempts. Two independent layers of authorization are used, which include the `RBACMiddleware` class at the API layer and Row-Level Security in PostgreSQL at the database layer; such that bypassing one layer does not lead to access. API hardening includes rate limiting, input validation with `express-validator`, parameterized queries to prevent SQL injection attacks, and security headers with `Helmet.js`. Isolation of network traffic is performed by using

Security Groups, Network Access Control Lists (NACLs), and VPC subnet isolation as described in Sections 4.3.3 and 4.3.4. At-rest encryption of all data is performed by AWS KMS (AES-256 with RDS), and transit encryption is performed using TLS 1.2 on the Application Load Balancer. An audit log is created by using the `audit.log` table at the application level and AWS CloudTrail. The full security control specifications and HIPAA compliance mapping are provided in Appendix F.

4.4 Database Design

This section presents the relational database schema designed for the proposed system. The design specifies the entity relationships, table structures, column definitions, and row-level security policies that enforce the access control requirements defined in Section 4.2. The schema comprises six tables — ‘users’, ‘patients’, ‘doctors’, ‘medical_records’, ‘appointments’, and ‘audit_log’ — each documented with its purpose, inter-table relationships, and the PostgreSQL row-level security policies that restrict data access to authorized users only.

4.4.1 Entity-Relationship Model

The database schema consists of six tables: ‘users’, ‘patients’, ‘doctors’, ‘medicalrecords’, ‘appointments’, and ‘auditlog’. Figure 4.5 presents the entity-relationship diagram.



Figure 4.4 — Entity-Relationship Diagram (Patient Data Management System)

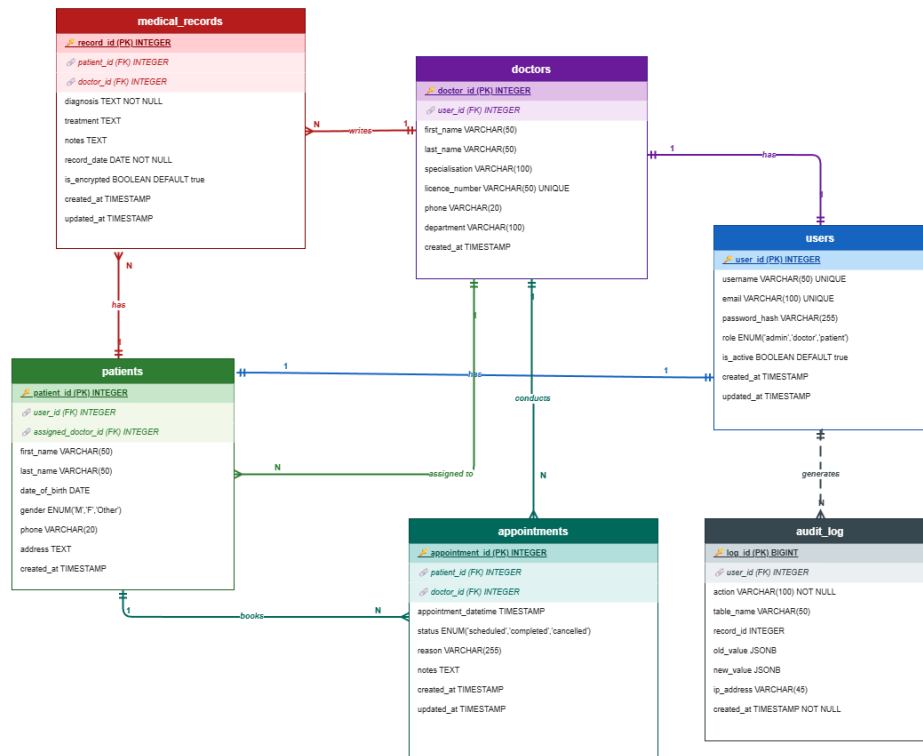


Figure 4.5 Entity-Relationship Diagram of the Patient Data Management System Database Schema

4.4.2 Database Schema

The database for the system consists of six tables namely users, patients, doctors, medical_records, appointments, and audit_log. In all the tables, all the primary keys make use of the UUID datatype to avoid enumeration attacks. Referential integrity in all the foreign key relationships is enforced by making sure that there can be no patient data without an existing user account and there is no entry in medical records for a nonexistent patient. All the timestamp fields have NOW() as their default value at the database level to prevent tampering of timestamps on the client side. The complete column-level schema for all six tables is provided in Appendix D.

4.4.3 Row-Level Security Policy

Row-level security on the PostgreSQL database is enabled for the ‘medical-Records’ table and ‘patients’ table in order to provide data isolation within the database

system, independent of application layer permissions. The following three policies are listed:

Policy 1 Doctor access to medical records: A doctor may only select, insert, or update records where ‘doctorId’ matches the current database session user. Records assigned to other doctors are invisible and inaccessible.

Policy 2 Patient access to own records: A patient may only select rows from ‘medicalRecords’ and ‘patients’ where ‘patientid’ matches their own user identifier. No patient can query another patient’s records regardless of application behaviour.

Policy 3 Admin exclusion from medical content: Admin database sessions are granted access to the ‘patients’ and ‘appointments’ tables only. Row-level security on ‘medicalRecords’ denies all access to sessions authenticated with the admin role, ensuring that administrative staff cannot view clinical record content even with direct database access.

This gives another level of protection; in case there is any vulnerability in the Node.js/Express application layer, the database will refuse to accept any queries that exceed their roles.

4.5 Interface Design

There are three unique interface views for the three types of users. These interface views have been made in such a way that there is role segregation, where all functionalities that a user needs to perform and can access are visible to him only.

4.5.1 Login Screen

All users access the system through a single login screen. The screen presents a username field, a password field, and a login button. On successful authentication, the system reads the role from the JWT token and redirects the user to the appropriate dashboard — Doctor Dashboard, Admin Dashboard, or Patient Portal. The wireframe layout for the login screen is shown in Figure 4.6.

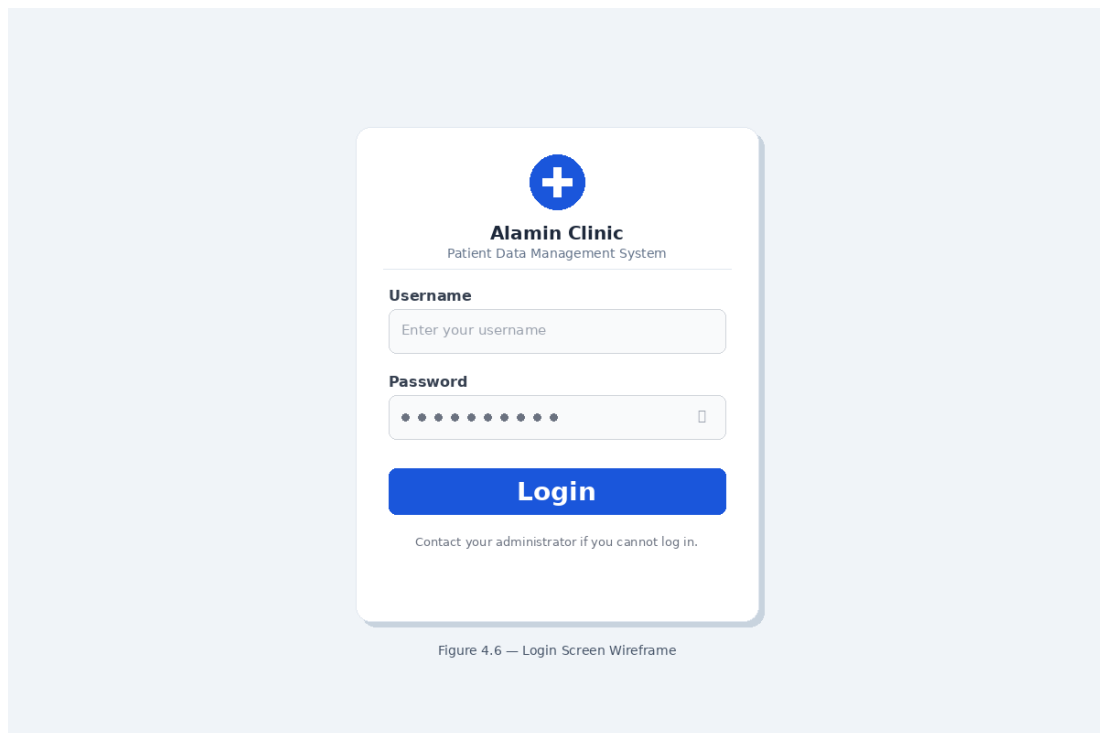


Figure 4.6 Login Screen Wireframe

4.5.2 Doctor Dashboard

The Doctor Dashboard presents the doctor with their assigned patient list and appointment schedule as the primary views. A navigation sidebar provides access to Patient Records and the appointment calendar. The patient list displays each patient's name, date of last visit, and a "View Records" action button. Selecting a patient opens the patient detail view, showing the full medical record history and a "New Record" button. The new record form presents fields for Diagnosis, Prescription, and Clinical Notes with a Submit button. Figure 4.7 presents the wireframe layout for the Doctor Dashboard.

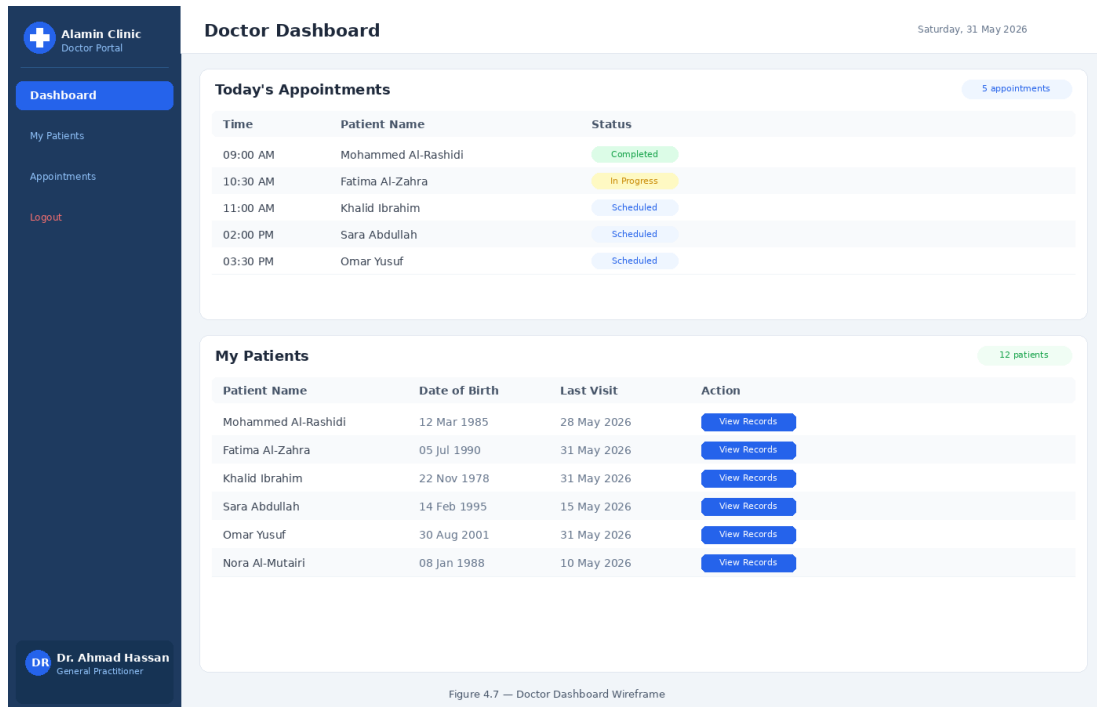


Figure 4.7 Doctor Dashboard Wireframe

4.5.3 Admin Dashboard

The Admin Dashboard centres on appointment management and patient registration. The navigation sidebar provides access to Patient Registration, Appointment Scheduling, and User Management. The appointment scheduling screen presents a calendar view with time slot selection. Admin selects a patient, selects a doctor, picks a date and time, and confirms the booking. The patient registration screen presents a form for new patient demographic details and doctor assignment. Figure 4.8 presents the wireframe layout for the Admin Dashboard.

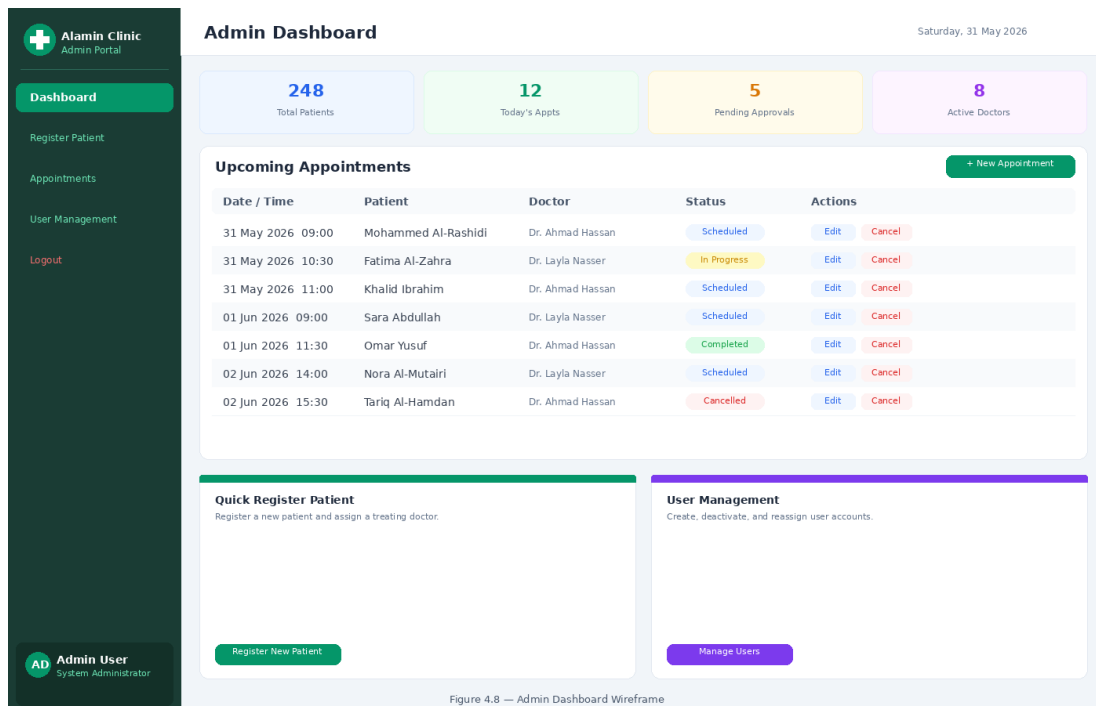


Figure 4.8 Admin Dashboard Wireframe

4.5.4 Patient Portal

Patient portal is a read-only portal. A patient can access his or her own medical records and appointments. There are no options to create, modify, or delete information within the patient's area. In the records view, user can will a list of records in chronological order that show date, physician, and patient's diagnosis. When a record is clicked on, it gives you full information including any prescribed medication or additional notes from the doctor. In the appointments view, future appointments are listed by date and time. Figure 4.9 presents the wireframe layout for the Patient Portal.

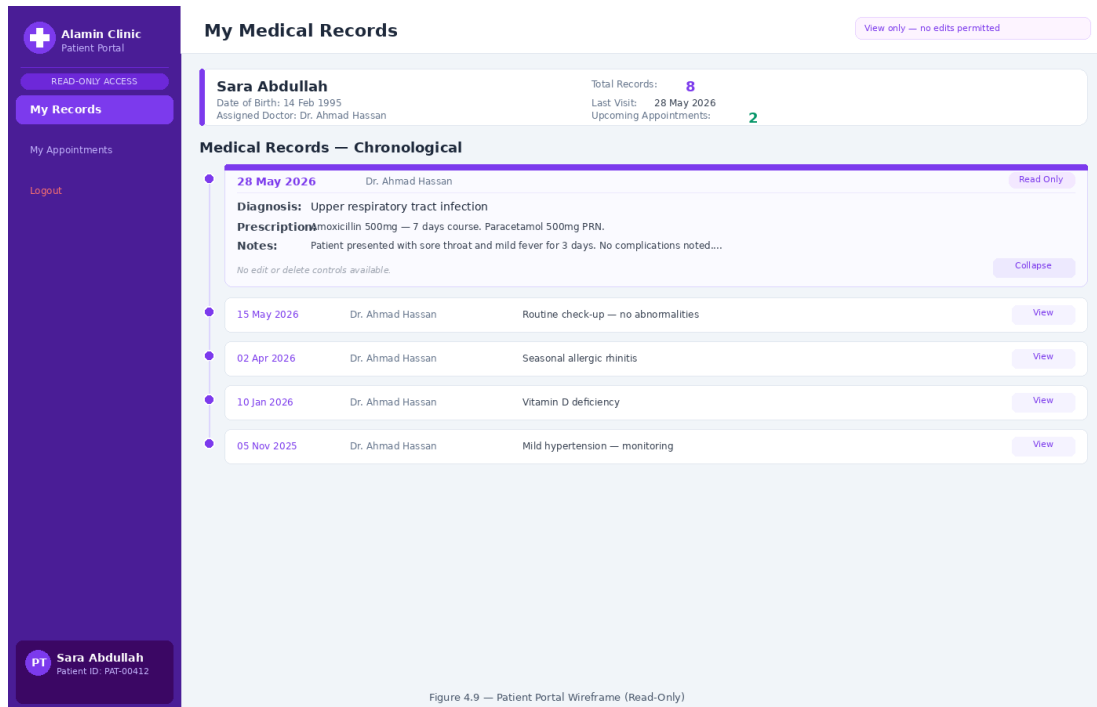


Figure 4.9 Patient Dashboard Wireframe

4.6 Chapter Summary

The chapter elaborates on the thorough requirement analysis and design of the Secure Cloud-Based Patient Data Management System, addressing the limitations outlined in the case study discussed in Chapter 2 and requirements described in Chapter 3. The architecture describes a very robust and resilient design, including the implementation of a three-tier architecture with six subnets on AWS VPC across two availability zones, along with layers of security groups, NACLs, and more precise IAM policies. In addition, a DevSecOps pipeline with automatic quality gates is introduced in order to ensure the left-shifted security approach to immediately block any deployment if it is found to be vulnerable. On the data layer, the PostgreSQL database employs a six-table schema with primary keys, foreign keys, an audit tracking table for HIPAA compliance, and separate RLS policy to ensure that role-based access control applies directly on the storage layer. Finally, the UI design defines the four different designs for the Login portal, Doctor Dashboard, Admin Dashboard, and Patient Portal, thus defining the architectural blueprint to support the implementation discussed in Chapter 5.